

Project Documentation: Deep learning for medical image diagnosis (AI Radiology Assistant)

1. System Overview & Architecture

AI Radiology Assistant is a multi-modal computer vision and natural language processing (NLP) system designed for deployment on Cloud platforms (such as Hugging Face Spaces). The architecture consists of three core components:

1. **Frontend UI Layer:** Built using **Gradio (Soft Theme Framework)** with embedded custom HTML5, CSS3 styling, and a dynamic inline SVG renderer.
2. **Inference Backend Layer:** Built on **TensorFlow/Keras** running optimized CPU-based sequential image tensors through three targeted Deep Learning architectures via Transfer Learning.
3. **Cognitive AI Layer:** Powered by the **Groq API** utilizing the **Llama 3.3 (70B)** parameter model to maintain stateful medical dialogue based on the outputs generated by the inference backend.

2. Technical Stack & Dependencies

The production build runs on Python 3.10+. All software modules are declared dynamically in requirements.txt:

- **tensorflow-cpu:** Used to load .h5 model architectures and perform forward propagation inference.
- **gradio:** Powers the event-driven web infrastructure and reactive input-output layouts.
- **groq:** High-throughput SDK provider utilized to communicate with the Llama 3.3 LLM.
- **numpy:** Manages efficient matrix operations, image resizing transformations, and dimension expansions.
- **pillow (PIL):** Handles backend multi-format image loading (JPG, PNG, DICOM/TIFF conversions) and color channel normalization.

3. Data Flow & Processing Pipeline

The execution sequence follows a strict deterministic pipeline upon user interaction:

```
```text [User Upload] → [Metadata Extraction] → [Image Resizing/RGB Conversion] |  
[Groq LLM Generation] ? [SVG Donut Chart] ? [Model Evaluation & Softmax Calibration]
```

### 3.1 Preprocessing Stage

Images captured via camera or uploaded directly are structured into standard 3-channel RGB arrays.

Pixel dimensions are evaluated and scaled depending on the user-selected scan category:

Brain MRI: 224 × 224

Chest X-Ray: 384 × 384

Chest CT: 380 × 380

Scaled matrices are normalized via the built-in `eff_preprocess` mapping functions to align pixel distributions with the original ImageNet checkpoint weights.

### 3.2 Post-Processing & Logit Calibration

To prevent mathematical artifacts such as unscaled confidence bounds from uncompiled sequential files, the core script routes outputs through a custom mathematical verification layer:

For multi-class pipelines, a clean wrapper forces compliance to 100% distribution across all target nodes.

### 4. Key Functions & Codebase Breakdown

`prep_image(img_path, target_size)`

Purpose: Reads images from temporary directories, drops unnecessary alpha channels, applies structural scaling transformations, and returns a 4-dimensional NumPy float array ready for inference.

`analyze_scan(scan_type, image)`

Purpose: Central routing hub. Triggers conditional execution branches to feed the processed image array into the corresponding target model object (`brain_model`, `xray_model`, or `ct_model`). Calculates top predictions and confidence thresholds.

`medical_chat(message, history, ...)`

Purpose: Orchestrates stateful memory with Groq endpoints. It dynamically injects a system prompt containing clinical restrictions alongside current patient metadata (Name, Age, Scan Type, Diagnostic Class, and Confidence Percentage) to enforce contextual awareness.

## 5. System Deployment & Configuration Guide

### 5.1 Environment Configuration

To initialize the Cognitive AI Layer, a valid API key must be provisioned in the root shell environment or within the platform configuration dashboard:

```
```bash
export GROQ_API_KEY="your_secure_groq_api_key_here"
```

6. UI/UX Design Specifications

- **Adaptive Dark/Light Themes:** Handled via inline Gradio component states bundled inside the settings drawer.
- **No-Dependency Visualizations:** Inline-generated SVG strings display distribution data efficiently without introducing secondary plotting overheads (such as Matplotlib/Seaborn thread blocking).
- **Session Memory Persistence:** In-memory tracking structures update a local historical database matrix on-the-fly, enabling users to log sequential diagnostics inside a real-time data table during the lifespan of the host session thread.